

使用 Amazon Bedrock AgentCore 建立 Agent with Knowledge Base

學員實作手冊

2026 年 8 月

使用 Amazon Bedrock AgentCore 建立 Agent with Knowledge Base

學員實作手冊

2026 年 8 月

AWS Professional Services

適用對象：工作坊參與者（不限技術背景） 環境：AWS Workshop Studio 提供之獨立帳號（Region：us-west-2）

預期成果：親手在 **Amazon Bedrock AgentCore** 建立一個 **Managed Knowledge Base**（全託管 **RAG**），餵入公開下載的 **AWS Well-Architected Framework** 文件。先在主控台操作與測試（純檢索、生成回應），再建立 **AgentCore Gateway** 以 **MCP connector** 暴露 Knowledge Base，最後用 **AgentCore Harness** 建立一個 Agent，把 Gateway 加為工具，在 Playground 完成問答 Demo。問答內容將直接對應餵進去的文件。

如何閱讀本手冊：凡是以灰底方框（> 引用區塊）呈現的內容，是「要你在畫面上輸入／貼上的具體值」或「要提出的問題」。方框以外的文字是操作說明與背景。方框內若出現以角括號標示的佔位符（例如 <你的英文名>），請替換成你自己的實際內容後再送出。

⚠ 命名一律用英文小寫字母 + 數字：AWS 資源名稱不接受中文（S3 bucket 須全小寫；Harness 只允許字母/數字/底線）。建議用你的英文名或代號，例如 `joe`。

目錄

1. 總覽與學習目標
2. 事前準備
3. 核心概念（三分鐘理解）
4. Step 1：下載範例文件（公開網站）
5. Step 2：建立 S3 儲存桶並上傳文件
6. Step 3：建立 Managed Knowledge Base（AgentCore 全託管 RAG）
7. Step 4：同步資料（Ingestion）
8. Step 5：測試檢索（回傳文件段落）
9. Step 6：開啟「生成回應」——讓模型根據知識庫回答（帶出處）

-
10. Step 7: 建立 AgentCore Gateway 並新增 Knowledge Base target
 11. Step 8: 用 AgentCore Harness 建立 Agent 並在 Playground 對話
 12. Step 9: 從本機 Terminal 呼叫 Harness Agent (選做)
 13. Step 10: 資源清理
 14. 常見問題與排解
 15. 附錄 A: Demo 問答題庫 (緊扣文件)
 16. 附錄 B: 進階延伸 (把 Agent 整合進應用程式)
 17. 附錄 C: 參考資源連結
-

1. 總覽與學習目標

本手冊帶你完成一件事：把一份公開的技術文件，變成 **Bedrock AgentCore** 的全託管知識庫，先在主控台操作它，再讓一個 **Agent** 查詢它、根據文件內容回答問題——全程只用瀏覽器。

你不需要自己架設向量資料庫，也不需要寫檢索或問答程式。**Amazon Bedrock AgentCore** 的 **Managed Knowledge Base** 幫你全託管向量儲存、資料切割（chunking）、嵌入（embedding）與檢索；**AgentCore Gateway** 把知識庫包裝成標準的 **MCP** 工具；**AgentCore Harness** 則讓你在 Console 宣告一個 Agent（選模型、寫指令、掛工具）並直接在 Playground 對話——完全不用寫程式。

完成本手冊後，你將能夠：

- 從公開網站下載文件，並上傳到 Amazon S3。
- 在 **AgentCore** → **Built-in tools** → **Knowledge Base** 建立一個 Managed Knowledge Base，完成資料同步。
- 在主控台測試面板進行純檢索（回傳來源段落）與生成回應（模型產生帶出處的答案）。
- 建立 **AgentCore Gateway**，新增 **Managed Knowledge Base target**，把知識庫暴露為 MCP 工具。
- 用 **AgentCore Harness** 在 Console 建立一個 Agent，把 Gateway 加為工具，在 Playground 對話——零程式碼。
- 正確清理資源，避免產生非預期費用。

核心體驗：整條 RAG 流程 — 「資料 → 全託管知識庫 → Gateway（MCP）→ Agent → 帶出處的答案」— 你會親手走過一遍，全程只用瀏覽器。

2. 事前準備

開始前請確認下列項目：

項目	說明
Workshop Studio 活動連結	由講師提供，用於登入你的專屬 AWS 帳號
瀏覽器	Chrome / Edge / Firefox 皆可，用於操作 AWS 主控台
網路連線	一般網路即可（下載文件、操作主控台）
Region	全程使用 us-west-2 （奧勒岡），登入後確認右上角 Region，請勿切換到其他 Region

本手冊全程在瀏覽器的 AWS 主控台完成，本機不需安裝任何軟體、不需命令列、不需設定憑證。用講師提供的 Workshop Studio 連結以個人 email 登入配發帳號即可開始。

3. 核心概念（三分鐘理解）

在動手前，先理解三個關鍵概念，有助於你理解每一步在做什麼。

3.1 什麼是 RAG（檢索增強生成）

大型語言模型（LLM）只知道它訓練時看過的資料，對「你的文件」一無所知。**RAG（Retrieval-Augmented Generation）**的做法是：先把你的文件建立成可檢索的知識庫，當使用者提問時，先「檢索」出最相關的段落，再把這些段落連同問題一起交給模型「生成」答案。這樣模型的回答就會根據你的文件，而且能附上來源。

3.2 為什麼用 AgentCore 的 Managed Knowledge Base

傳統做 RAG，你要自己準備向量資料庫、寫切割與嵌入程式、維護檢索邏輯。**Bedrock AgentCore 的 Managed Knowledge Base** 把這些全部託管：預設連 embedding 模型都不用選（採 service-managed 模型），也沒有向量資料庫需要你佈建或營運。你只要指定「資料在哪裡（S3）」，其餘由 AgentCore 處理。

3.3 從「主控台測試」到「接上 Agent」的三種深度

本手冊會依序體驗三種深度，一層比一層接近真實應用，且全部在 Console 完成：

深度	怎麼做	你會看到
純檢索（Step 5）	KB 測試面板，關閉生成回應	文件裡最相關的原始段落與來源
生成回應（Step 6）	KB 測試面板，開啟生成回應	模型合成的自然語言答案 + 引用出處
接上 Agent （Step 7–8）	Gateway 把 KB 變 MCP 工具，Harness Agent 掛上並在 Playground 對話	Agent 自主決定何時檢索、如何根據結果回答

名詞速記：**Gateway** 把 Managed KB 包裝成 **MCP** 工具（`Retrieve` = 單次檢索、`AgenticRetrieveStream` = 多步規劃並合成帶 citation 的答案）；**Harness** 是 AgentCore 的「免寫程式 Agent」——在 Console 宣告 model + 指令 + 工具就能跑，由 AgentCore 處理整個 agent loop。

4. Step 1: 下載範例文件（公開網站）

本手冊使用 **AWS Well-Architected Framework** 系列文件作為知識庫內容——AWS 官方公開、免費，內容明確，很適合驗證「答案是否來自文件」。

請從下列公開網址下載 PDF（至少下載前兩份，其餘可選擇性加入）：

文件	下載連結
Framework（總覽，必下載）	https://docs.aws.amazon.com/pdfs/wellarchitected/latest/framework/wellarchitected-framework.pdf
Cost Optimization Pillar（必下載）	https://docs.aws.amazon.com/pdfs/wellarchitected/latest/cost-optimization-pillar/wellarchitected-cost-optimization-pillar.pdf
Security Pillar（選擇性）	https://docs.aws.amazon.com/pdfs/wellarchitected/latest/security-pillar/wellarchitected-security-pillar.pdf
Reliability Pillar（選擇性）	https://docs.aws.amazon.com/pdfs/wellarchitected/latest/reliability-pillar/wellarchitected-reliability-pillar.pdf
Performance Efficiency Pillar（選擇性）	https://docs.aws.amazon.com/pdfs/wellarchitected/latest/performance-efficiency-pillar/wellarchitected-performance-efficiency-pillar.pdf

在瀏覽器貼上網址即可下載，並把檔案集中放在一個資料夾（例如桌面的 `wa-docs/`）。

本步驟檢核點：本機已有至少 2 份 Well-Architected PDF。

5. Step 2: 建立 S3 儲存桶並上傳文件

Managed Knowledge Base 會從 S3 讀取你的文件，因此先把文件放上 S3。

1. 前往 **S3** 主控台 → **Create bucket**。
2. Bucket 名稱取易於辨識者，例如 `agentcore-kb-<你的英文名>`（名稱需全球唯一、全小寫）。
3. Region 確認為 **us-west-2**，其餘保留預設，建立。
4. 進入該 bucket，**Upload** → **Add files**，把 Step 1 下載的 PDF 全部上傳。

本步驟檢核點：S3 bucket 內可看到你上傳的 PDF。請記下 bucket 名稱，下一步會用到。

6. Step 3: 建立 Managed Knowledge Base (AgentCore 全託管 RAG)

這是本手冊的核心。你將在 **AgentCore** 建立全託管知識庫。

1. 前往 **Amazon Bedrock AgentCore** → **Built-in tools** → **Knowledge Base**。
2. 選擇 **Create Managed Knowledge Base**。
3. 填寫 Knowledge Base 名稱，例如：

```
agentcore-wa-kb-<你的英文名>
```

1. (可選) 展開 **Additional configurations**：- **Embedding model type**：保留 **Managed** (預設) —— 由服務自動選用 embedding 模型，無需設定。

- **IAM permissions**：選 **Create and use a new service role** (建議)。
- **KMS encryption**：保留 AWS managed key 預設值。

1. 在 **Data source** 區塊：- Data source name 填 `wa-s3-source`。

- Data source type 選 **Amazon S3**。
- 設定連線指向 Step 2 的 bucket：

```
s3://agentcore-kb-<你的英文名>
```

1. (可選) 展開 **Content parsing and chunking**：- Parsing strategy 保留 **Managed parser**。

- Chunking 選 **Default chunking** (建議)。

1. 選擇 **Create Knowledge Base**，等待建立完成 (約 2-5 分鐘)。

本步驟檢核點：Knowledge Base 狀態為可用 (Available)，且底下掛著一個指向你 S3 的 data source。請記下 **Knowledge Base** 名稱/ID，Step 7 會用到。

7. Step 4: 同步資料 (Ingestion)

建立完成後，資料還沒被讀進知識庫，需要執行一次同步 (ingestion)，AgentCore 才會去讀 S3、切割、嵌入並建立索引。

1. 在該 Knowledge Base 頁面，找到 data source。
2. 選取 data source，點擊 **Sync**。

3. 等待同步完成（依文件大小與數量，通常數分鐘）。狀態會由進行中轉為完成，並顯示已處理的文件數。

本步驟檢核點：Sync 狀態顯示完成，且處理的來源文件數與你上傳的份數一致。

提醒：日後若更新 S3 的文件，需要再次 Sync，知識庫才會反映新內容。

8. Step 5：測試檢索（回傳文件段落）

先體驗第一種深度：純檢索——看看知識庫從文件裡撈出什麼。

1. 在 Knowledge Base 頁面，點選 **Test**（測試）開啟右側測試面板。
2. 關閉面板中的「**Generate responses**（生成回應）」開關（這樣只回傳檢索段落）。
3. 輸入一個答案明確在文件內的問題，例如：

Well-Architected Framework 的六大支柱是什麼？

1. 點 **Run**。觀察回傳結果：會列出最相關的原始段落，並標示來源。

本步驟檢核點：測試面板能回傳與問題相關的文件段落，且來源指向你上傳的 Well-Architected 文件。

9. Step 6：開啟「生成回應」——讓模型根據知識庫回答（帶出處）

再體驗第二種深度：讓模型讀取檢索結果、生成自然語言答案並附出處。

1. 在同一個測試面板，開啟「**Generate responses**（生成回應）」開關。
2. （若面板要求選擇模型）選一個基礎模型，例如 Claude 系列。
3. 輸入問題，比較這次的回答形式：

Well-Architected Framework 的六大支柱分別是什麼？請簡述每一支柱。

1. 觀察：這次會回傳一段完整的自然語言答案，並在答案旁標示引用來源（**citation**），點擊可看到對應的文件段落。

本步驟檢核點：開啟生成回應後，能得到帶引用出處的自然語言答案，且來源可追溯到你上傳的文件。

到這裡，你已在主控台完成一次完整的 RAG 問答。接下來 Step 7–8 會把同一個知識庫接到一個真正的 Agent，一樣全程在 Console。

10. Step 7: 建立 AgentCore Gateway 並新增 Knowledge Base target

現在把知識庫透過 Gateway 暴露成 MCP 工具，讓 Agent 能呼叫。全程在主控台完成，精靈分成四頁：**Define gateway details** → **Configure Inbound Identity** → **Add targets** → **Review and create**。

前往 **Amazon Bedrock AgentCore** → **Gateway**，選擇 **Create gateway**，依序完成四頁：

第 1 頁: Define gateway details

1. **Gateway name**: 例如 `wa-kb-gateway-<你的英文名>`（可保留系統產生的預設名）。
2. **Permissions** → **IAM permissions**: 選 **Create default role**（主控台會自動建立一個帶所需權限的新角色）。Policy / KMS key / WAF 皆為 optional，保留預設即可。
3. 按 **Next**。

第 2 頁: Configure Inbound Identity（控制「誰能呼叫這個 Gateway」）

1. **Inbound Auth type**: 選 **Use IAM permissions**（以 AWS SigV4 驗證，無需其他設定）。
2. 按 **Next**。

第 3 頁: Add targets（定義這個 Gateway 要暴露哪些工具）

1. **Select a target protocol**: 選 **MCP target**。
2. **Target name**: 例如 `managed-kb`（可保留預設）。
3. **Passthrough**: 保留 **Do not use passthrough - default aggregated**（預設）。
4. **Target type**: 選 **Connectors - new**（以內建整合連接器定義 target）。
5. **Pre-configured targets**: 選 **Knowledge Bases (KB)**。
6. **Runtime retrieval type**: 選 **Standard retrieval**。
7. **Knowledge Base (KB)**: 從下拉選你在 Step 3 建立的 KB（`agentcore-wa-kb-<你的英文名>`）。
8. **Outbound Auth configurations**: 選 **IAM Role**。
9. （可選）**Tool description**: 可填「Retrieve data from this Knowledge Base」幫助 Agent 判斷何時用此工具。
10. 按 **Next**。

第 4 頁：Review and create

1. 檢視摘要（應顯示 Inbound Auth type: IAM、Target type: knowledgeBase、Protocol type: MCP），確認無誤後選 **Create Gateway**。
2. 建立後在 Gateway 詳情頁記下 **Gateway 名稱 / endpoint**（Step 8 掛工具時要選它）。

本步驟檢核點：Gateway 狀態為就緒，且底下的 target 狀態為就緒。target 會暴露 `Retrieve` 與 `AgenticRetrieveStream` 兩個工具（工具名以 target 名為前綴，如 `managed-kb-    Retrieve`）。

11. Step 8：用 AgentCore Harness 建立 Agent 並在 Playground 對話

最後一步：用 **AgentCore Harness** 在 Console 建立一個 Agent，把 Step 7 的 Gateway 加為工具，然後直接在 Playground 對話。完全不用寫程式。

Harness 是 AgentCore 的「免寫程式 Agent」——你只需在 Console 宣告 model、system prompt、tools，AgentCore 就會替你跑整個 agent loop（推理、選工具、執行、回覆）。

11.1 建立 Harness Agent

1. 前往 **Amazon Bedrock AgentCore → Harness**。
2. 選擇 **Advanced create Harness**（進階建立，才能設定 system prompt 與掛工具）。
3. **Harness 名稱**：例如 `waragent_<你的英文名>`（建立後不可改名）。⚠️ 命名規則與 Gateway 不同：只允許英文字母、數字、底線 `_`（不可用連字號 `-`，不可用中文），須以字母開頭、上限 40 字。例如 `waragent_joe`。
4. **Model**：Model source 選 **Bedrock**，挑一個 Claude 系列模型（如 Claude Sonnet）。
5. **System prompt**：填入下列指令，要求 Agent 先檢索、附出處、查無則誠實說明：

你是 AWS Well-Architected 助理。回答前務必先使用知識庫工具檢索，只根據檢索到的內容回答，並附上來源。若知識庫中找不到答案，請誠實說明沒有相關資訊，不要自行編造。

1. 在 **Tools** 區塊，新增 **Gateway connection**，選擇 Step 7 建立的 Gateway（`wa-kb-gateway-<你的英文名>`）。這會把 `Retrieve` / `AgenticRetrieveStream` 掛給 Agent。
2. （可選）**Memory**：可開啟 managed memory 體驗多輪對話記憶。
3. 選擇 **Create Harness**，等待狀態變為 **Ready**（綠色）。

11.2 在 Playground 做 Demo

1. 在 Harness 詳情頁選擇 **Test in playground**（或從 Harness 頁面進入 Playground）。
2. 依序輸入下列問題，觀察 Agent 何時呼叫知識庫工具、以及答案如何引用文件（完整題庫見附錄 A）：

Well-Architected Framework 的六大支柱分別是什麼？

請比較可靠性與成本最佳化支柱在設計取舍上的差異，並附出處。

1. 再問一個文件中不存在的问题，驗證 Agent 會誠實說「查無資料」而非編造：

這份文件對量子運算的區塊鏈挖礦效能有什麼建議？

1. 展開「**Agent trace**」看內部運作：每則回答上方可展開 Agent trace，看到 Agent 呼叫了 **Managed-Kb Retrieve** 工具，並可檢視該次呼叫的 **Input**（`retrievalQuery` 文字）與 **Output**（`retrievalResults`，含每段的 `score` 與來源 `metadata`，例如 `_document_title: wellarchitected-framework.pdf`）。這讓你親眼看到「Agent 何時、如何查知識庫」，而不只是看到最終答案。

本步驟檢核點：Playground 中 Agent 會呼叫知識庫工具；文件內問題回答正確且附來源；文件外問題誠實表示查無資訊。

🎉 恭喜！你已經完成整條 **RAG** 流程：從一份公開文件，建立全託管知識庫，接上一個會查資料、附出處、且不亂編的 Agent。

接下來可以挑戰 Step 9（選做）：用程式碼從本機呼叫這個 Agent，體驗真正整合進應用程式的模式。

12. Step 9：從本機 Terminal 呼叫 Harness Agent（選做）

前面所有操作都在瀏覽器完成。這個選做步驟讓你體驗「用程式呼叫 Agent」——從自己的電腦（Mac / Windows / Linux）透過 Python + boto3 呼叫剛才建好的 Harness Agent，感受真正整合進應用程式的模式。

⚠️ 本步驟需要本機有 Python 3.9+ 環境。若你的電腦沒有 Python，可跳過此步。

12.1 建立專案與虛擬環境

打開 Terminal（Mac/Linux）或 PowerShell（Windows），執行：

```
mkdir agentcore-test && cd agentcore-test
python3 -m venv .venv
source .venv/bin/activate
```

Windows 使用者改用：

```
python -m venv .venv
.venv\Scripts\activate
```

12.2 安裝新版 boto3

AgentCore Harness 的 API 需要較新的 boto3 版本，請安裝：

```
pip install --upgrade "boto3>=1.43.70"
python -c "import boto3; print(boto3.__version__)"
```

確認輸出的版本號 `>= 1.43.70`。

12.3 取得臨時 AWS 憑證

1. 回到瀏覽器的 **Workshop Studio** 頁面。
2. 在左側導覽找到 **Get AWS CLI credentials**（或類似文字）。
3. 複製畫面上提供的臨時憑證（包含 `AWS_ACCESS_KEY_ID`、`AWS_SECRET_ACCESS_KEY`、`AWS_SESSION_TOKEN`）。

12.4 在 Terminal 中貼入憑證

將上一步複製的憑證貼到 Terminal 中，格式如下：

```
export AWS_ACCESS_KEY_ID="<貼上你的 Access Key>"
export AWS_SECRET_ACCESS_KEY="<貼上你的 Secret Key>"
export AWS_SESSION_TOKEN="<貼上你的 Session Token>"
export AWS_DEFAULT_REGION="us-west-2"
```

Windows PowerShell 改用 `$env:AWS_ACCESS_KEY_ID = "..."` 語法。

12.5 建立 invoke.py 並貼上範例程式碼

1. 回到 Harness 詳情頁，點選 **View invocation code**。
2. 選擇 **Python** 分頁，複製完整的範例程式碼。
3. 在專案資料夾中建立 `invoke.py`，貼上複製的程式碼。

4. 修改程式碼中的 `runtimeSessionId` ——必須 ≥ 33 個字元，例如：

```
i-love-aws-and-this-workshop-is-great-123
```

💡 提示：`runtimeSessionId` 是用來識別一輪對話的 ID。使用相同 ID 可延續同一個 session；換一個新的 ID 則開啟新對話。

12.6 執行程式

```
python invoke.py
```

你應該會在 Terminal 中看到 Agent 的回覆，跟 Playground 中看到的一樣——只是這次是透過程式碼呼叫。

本步驟檢核點：Terminal 中成功收到 Harness Agent 的回覆，內容引用了知識庫文件。

恭喜！你已經體驗了「從 Console 到程式碼」的完整路徑。這代表你可以把同樣的邏輯嵌入任何 Web 後端、Line Bot、Slack Bot 或其他應用程式中。

13. Step 10：資源清理

Workshop 結束後，請依序清理以避免持續計費（全部在主控台操作）：

1. **Harness**：於 AgentCore → Harness 刪除你建立的 Agent（連同其背後自動建立的 runtime）。
2. **Gateway**：於 AgentCore → Gateway 刪除 `managed-kb` target，再刪除 Gateway。
3. **Knowledge Base**：於 AgentCore → Knowledge Base 刪除你建立的 KB（含 data source）。
4. **S3**：清空並刪除 Step 2 建立的 bucket。
5. **IAM** 角色：刪除為此 workshop 自動建立的相關 service role。

Workshop Studio 活動結束後通常會自動回收整個帳號，但當場清理仍為良好習慣。

本步驟檢核點：AgentCore 中已無你建立的 Harness、Gateway 與 Knowledge Base；S3 bucket 已刪除。

14. 常見問題與排解

錯誤訊息／現象	原因	解法
測試檢索回傳空白	Sync 未完成或 S3 路徑錯誤	確認 Step 4 Sync 完成、S3 路徑與 bucket 正確
Sync 失敗	文件格式不支援或角色權限不足	確認為 PDF/文字類文件；確認 KB service role 可讀該 S3
建立 Gateway/KB 時 IAM 角色錯誤	帳號無法建立 service role	改選既有角色，或向現場助教尋求協助
Gateway target 找不到 Knowledge Base 選項	KB 尚未就緒	確認 KB 為 Available 後重試
Harness 掛不上 Gateway 工具	Gateway 尚未就緒	確認 Gateway 與 target 狀態皆為就緒
建立資源時名稱被拒	用了中文或不合規字元	資源名一律用英文小寫 + 數字；Harness 只允許字母/數字/底線
Agent 回答未引用文件 / 自行編造	system prompt 未強制檢索	沿用 11.1 的 system prompt
本機 invoke.py 報 <code>ExpiredTokenException</code>	臨時憑證已過期	回到 Workshop Studio 重新複製憑證並貼入 Terminal
本機 invoke.py 報 <code>ModuleNotFoundError: boto3</code>	虛擬環境未啟用或 boto3 未安裝	確認已 <code>source .venv/bin/activate</code> 且已 <code>pip install boto3</code>
本機 invoke.py 報 <code>ValidationException on sessionId</code>	runtimeSessionId 不足 33 字元	換一個 ≥ 33 字元的字串

排解技巧：遇到錯誤時，先確認 Region 是否為 us-west-2、KB 是否 Available、Sync 是否完成。若屬於權限層級的問題（例如無法建立角色），請向現場助教尋求協助。

15. 附錄 A：Demo 問答題庫（緊扣文件）

以下問題的答案都能在 Well-Architected 文件中找到，適合示範「答案來自文件、且可附出處」。Step 5/6（KB 測試面板）與 Step 8（Harness Playground）都適用。

基礎（單次檢索即可）

- Well-Architected Framework 的六大支柱分別是什麼？
- 成本最佳化支柱的設計原則有哪些？
- 安全性支柱建議如何管理身分與存取？（需下載 Security Pillar）
- 可靠性支柱如何定義「從故障中恢復」？（需下載 Reliability Pillar）

進階（多步、跨文件、合成答案）

- 請比較「可靠性」與「成本最佳化」兩支柱在設計取舍上的差異，並附出處。
- 一個新創團隊資源有限，依 Well-Architected 建議，應優先關注哪些支柱與實務？
- 綜合多個支柱，說明如何在提升效能的同時控制成本。

驗證誠實性（文件外問題，**Agent** 應說查無資料）

- 這份文件對量子運算的區塊鏈挖礦效能有什麼建議？
- Well-Architected 對於在火星資料中心的部署有何規範？

小技巧：文件外問題是 RAG 最有說服力的一刻——它展示 Agent「查得到才答、查不到就誠實說」，這正是知識庫接上 Agent 的價值。

16. 附錄 B：進階延伸（把 **Agent** 整合進應用程式）

本手冊的動手範圍到 Playground 對話為止。若你想更進一步，把這個 Agent 整合進自己的網站或後端，AgentCore 提供現成的方式（這裡僅作概念介紹）：

- **View invocation code**：Harness 詳情頁的 **View invocation code** 可直接取得 Python / TypeScript / JavaScript 的呼叫程式碼（已自動填入你的 Harness ARN），貼進你的應用程式即可呼叫。每個 Harness 建好時，AgentCore 已在背後佈建好可呼叫的 endpoint，你不需要自己部署伺服器。
- **Harness Memory**：開啟 managed memory，讓 Agent 跨 session 記得使用者偏好與對話脈絡。
- **AgentCore Identity / Observability**：加入使用者驗證與追蹤，把 Demo 推向 production。

注意：要在自己的 AWS 帳號用程式呼叫 Harness，需要對應的 IAM 權限與模型存取權限。本 workshop 的配發帳號主要供主控台操作與 Playground 體驗使用。

參考資源：

- Amazon Bedrock AgentCore 開發者文件：<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/>
 - AgentCore Harness：<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/harness.html>
 - Managed Knowledge Base as Connector Target：<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-target-connector-managed-kb.html>
-

17. 附錄 C：參考資源連結

以下是本 Workshop 涉及的 AWS 服務文件與學習資源，供課後進一步探索。

Amazon Bedrock

資源	連結
Amazon Bedrock 官方文件	https://docs.aws.amazon.com/bedrock/latest/userguide/
Amazon Bedrock API Reference	https://docs.aws.amazon.com/bedrock/latest/APIReference/
Bedrock 支援的基礎模型列表	https://docs.aws.amazon.com/bedrock/latest/userguide/models-supported.html
Bedrock Pricing	https://aws.amazon.com/bedrock/pricing/
Bedrock Workshop (官方 Hands-on)	https://catalog.us-east-1.prod.workshops.aws/workshops/a4bdb007-5600-4368-81c5-ff5b4154f518/en-US

Amazon Bedrock AgentCore

資源	連結
AgentCore 開發者指南	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/
AgentCore Harness	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/harness.html
AgentCore Runtime	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime.html
AgentCore Gateway	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html
AgentCore Memory	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/memory.html
AgentCore Identity	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/identity.html
Managed Knowledge Base	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/knowledge-bases.html
Gateway - KB Connector Target	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-target-connector-managed-kb.html
AgentCore Built-in Tools 總覽	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/tools.html

資源	連結
AgentCore Observability	https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html

Amazon SageMaker AI

資源	連結
Amazon SageMaker AI 官方文件	https://docs.aws.amazon.com/sagemaker/latest/dg/
SageMaker JumpStart (模型中心)	https://docs.aws.amazon.com/sagemaker/latest/dg/studio-jumpstart.html
SageMaker HyperPod	https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-hyperpod.html
SageMaker Studio	https://docs.aws.amazon.com/sagemaker/latest/dg/studio.html
SageMaker Inference (模型部署)	https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model.html
SageMaker Pricing	https://aws.amazon.com/sagemaker/pricing/

其他相關資源

資源	連結
AWS Well-Architected Framework	https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
AWS Generative AI 總覽頁	https://aws.amazon.com/generative-ai/
Strands Agents (開源 Agent 框架)	https://github.com/strands-agents/sdk-python
Model Context Protocol (MCP) 規範	https://modelcontextprotocol.io/
boto3 Bedrock AgentCore Runtime Client	https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock-agentcore-runtime.html

流程速覽

步驟	你要做的事	完成標準
Step 1 下載文件	從公開網址下載 Well-Architected PDF	本機有 ≥2 份 PDF
Step 2 上傳 S3	建 bucket、上傳文件	S3 可見文件
Step 3 建 KB	AgentCore → Knowledge Base 建 Managed KB	KB 為 Available
Step 4 同步	Sync data source	同步完成，文件數相符
Step 5 測檢索	關閉生成回應，提問	回傳相關段落與來源
Step 6 生成回應	開啟生成回應，提問	得到帶出處的自然語言答案
Step 7 建 Gateway	Console 建 Gateway + Knowledge Base target	Gateway/target 就緒
Step 8 建 Agent	Harness 建 Agent + 掛 Gateway，Playground 對話	答案引用文件、文件外誠實說明
Step 9 地端呼叫（選做）	Python + boto3 從 Terminal 呼叫 Harness	Terminal 收到 Agent 回覆
Step 10 清理	刪 Harness/Gateway/KB/S3/角色	資源已清除